

Hierarchical Ordinal Radon Risk Modeling Using Hamiltonian Monte Carlo and NUTS

Noah Simonds
University of Virginia
Bayesian Data Analysis (STAT 6440)

December 16, 2025

Abstract

We study Hamiltonian Monte Carlo (HMC) and the No-U-Turn Sampler (NUTS) in the context of a hierarchical ordinal regression model for radon potential in Oregon. The data consist of $N \approx 120,000$ geologic polygons, each assigned a final radon potential rank in $\{1, 2, 3\}$ and grouped into $J = 66$ geological terranes. We specify a Bayesian ordered logit model with terrane-level random intercepts, fit the model using NUTS implemented in JAX via the `blackjax` library, and evaluate sampler performance with standard diagnostics. We then use the posterior draws to compute terrane-level posterior probabilities for each radon rank and to identify terranes with elevated probability of the highest radon risk category. The results show that an automatically tuned NUTS implementation can efficiently explore this posterior and produce useful summaries for radon risk mapping.

Introduction

Hamiltonian Monte Carlo (HMC) is a standard method for sampling from complex Bayesian posterior distributions, especially in hierarchical models with many parameters [4]. By using gradients of the log posterior to define approximate Hamiltonian dynamics, HMC can move long distances in parameter space with relatively little random-walk behavior and can therefore produce more efficient Markov chains than simple Metropolis–Hastings updates [2, 3]. However, basic HMC requires the user to specify both a step size and an integration time (or equivalently, a number of leapfrog steps). Poor choices of these tuning parameters lead either to very low acceptance rates or to highly correlated samples, and manual tuning becomes increasingly difficult as model dimension and posterior complexity grow.

The No-U-Turn Sampler (NUTS) was introduced by Hoffman and Gelman to address these tuning problems. NUTS adaptively chooses the integration time on each iteration by simulating Hamiltonian dynamics forward and backward in time until the simulated trajectory begins to turn back toward its starting point. This no-U-turn diagnostic provides a local

stopping rule that avoids both very short and excessively long trajectories. NUTS also combines this rule with adaptive step-size selection based on primal–dual averaging during a warmup phase. The resulting algorithm automatically selects both the step size and an effective path length while preserving detailed balance with respect to the target posterior. Empirical studies show large gains in effective sample size per gradient evaluation, especially for hierarchical models with strong posterior correlations and funnel-shaped geometries [1].

Later work has emphasized the interaction between HMC/NUTS and model parameterization in hierarchical settings, including non-centered parameterizations for varying-intercept and varying-slope models, systematic use of diagnostic quantities such as effective sample size and split $\hat{R}$, and practical guidelines for prior specification in multilevel models. These developments highlight that sampler performance depends strongly on posterior geometry and on the choice of parameterization; with appropriate reparameterizations and an adapted mass matrix, NUTS performs reliably in many multilevel regression and generalized linear mixed-model applications [4, 3].

Background and Literature Review

Our case study focuses on a radon potential dataset for the state of Oregon produced by the Oregon Department of Geology and Mineral Industries (DOGAMI). The primary feature class consists of $N = 120,049$ geologic polygons across Oregon, each assigned a final radon potential rank (`FIN_RN_RNK`) based on three major geologic factors: geologic materials, aerial radiometric measurement averages, and indoor air radon measurement averages [8]. Additional attributes include a geologic map unit label (`MAP_UNIT_L`), a generalized geologic unit (`GEO_GENL_U`), a terrane group (`TERRANE_GR`), and several intermediate rank variables (`GEOL_RANK`, `EU_RANK`, `INDOOR_RNK`). The radon potential ranks are intended to show where geologic conditions may contribute to an indoor radon hazard.

Radon is a known health hazard, and spatial variation in radon potential is tied to geology, uranium content, and surficial processes. Existing radon maps often rely on deterministic overlays of geology, radiometrics, and limited indoor measurements. A Bayesian hierarchical model provides a way to combine these sources of variation, borrow strength across geologic units, and quantify uncertainty in risk classifications at the terrane level. In particular, a multilevel ordinal regression can represent variation in radon risk across geologic terranes while allowing partial pooling, which stabilizes estimates in sparsely observed units and regularizes extreme risk estimates.

Automatic differentiation frameworks such as JAX make it straightforward to implement HMC and NUTS outside of traditional compiled probabilistic programming systems while still using exact gradients. In this analysis, we require posterior inference in a hierarchical ordinal model with $N \approx 120,000$ polygons nested within $J = 66$ geological terranes. The outcome variable `FIN_RN_RNK` is an ordinal risk category with three levels, and the posterior distribution includes terrane-specific intercepts, hyperparameters for their population distribution, and ordered cutpoints for the ordinal likelihood. Because of partial pooling and the ordinal threshold structure, the posterior is expected to exhibit substantial dependence

across parameters, making NUTS an attractive option relative to simpler random-walk or Gibbs samplers.

To perform inference in this model, we implement NUTS using the `blackjax` library in `JAX`. We write the log joint density of the hierarchical model in `JAX` and rely on automatic differentiation to obtain exact gradients with respect to all parameters. We then run an adaptive NUTS procedure with an initial warmup phase in which the step size and diagonal mass matrix are tuned by dual averaging and variance adaptation following [1], and we collect HMC samples under the adapted settings.

Radon mapping efforts in the United States typically combine geologic information, airborne radiometric surveys, and indoor radon measurements to produce qualitative and quantitative assessments of radon potential. State-level agencies such as DOGAMI assign geologic units to risk categories based on expert judgment and empirical evidence from measurements, yielding categorical maps that indicate relative radon potential across space. These maps are widely used for communication and planning, but their uncertainty is often implicit rather than quantified.

Bayesian hierarchical models provide a principled framework for representing radon risk across spatial or geologic units while allowing partial pooling. In such models, unit-level responses (e.g., radon ranks or concentrations) are linked to group-level effects (e.g., terrane or geologic unit) through a regression structure, and group effects are modeled as draws from a common population distribution. This structure stabilizes estimates in sparsely observed units and yields posterior distributions for group-level quantities of interest, such as the probability that a given terrane falls in a high-radon category. Posterior predictive checks can then be used to assess whether the model reproduces key features of the observed radon distribution.

HMC exploits gradient information to make long-distance proposals that follow level sets of the posterior, reducing random-walk behavior. NUTS further removes the need to hand-tune the trajectory length in HMC by stopping automatically when the simulated path starts to turn back, and it uses an adaptive scheme to tune the step size and mass matrix during warmup. These algorithms are now widely implemented in general-purpose Bayesian software and are well suited to the multilevel ordinal radon model considered here.

Methods

Data and radon rank response

The DOGAMI radon potential dataset consists of $N = 120,049$ geologic polygons across Oregon. Each polygon corresponds to a geologic map unit and is assigned a final radon potential rank (`FIN_RN_RNK`) based on three major geologic factors: geologic materials, aerial radiometric measurement averages, and indoor air radon measurement averages. The dataset includes several categorical and numeric attributes; the variables used directly in this analysis are:

- **TERRANE_GR**: a categorical geologic terrane group with $J = 66$ distinct levels.
- **FIN_RN_RNK**: final radon potential rank in $\{1, 2, 3\}$, with 1 denoting lowest radon potential and 3 denoting highest radon potential.

We restrict attention to polygons with non-missing **FIN_RN_RNK** and **TERRANE_GR**. The resulting analysis dataset has $N = 120,049$ observations and $J = 66$ terranes.

The marginal distribution of **FIN_RN_RNK** is highly unbalanced: approximately 78% of polygons are rank 1, 16% are rank 2, and 6% are rank 3. Radon potential varies systematically with geologic terrane. Some terranes are almost entirely in rank 1, while others show a higher proportion of rank 2 and rank 3 polygons, motivating a hierarchical model with terrane-specific effects on radon risk.

Hierarchical ordinal model

Let $y_i \in \{1, 2, 3\}$ denote the final radon rank for polygon i and let $j(i) \in \{1, \dots, J\}$ index the terrane group, with $J = 66$. We model y_i using an ordered logistic regression with terrane-level intercepts:

$$\Pr(y_i \leq c \mid \eta_i) = \text{logit}^{-1}(\kappa_c - \eta_i), \quad c = 1, 2,$$

where $\eta_i = \alpha_{j(i)}$ is the linear predictor for polygon i and (κ_1, κ_2) are ordered cutpoints with $\kappa_1 < \kappa_2$. The category probabilities are

$$\begin{aligned} p_{i1} &= \Pr(y_i = 1) = \text{logit}^{-1}(\kappa_1 - \eta_i), \\ p_{i2} &= \Pr(y_i = 2) = \text{logit}^{-1}(\kappa_2 - \eta_i) - \text{logit}^{-1}(\kappa_1 - \eta_i), \\ p_{i3} &= 1 - \text{logit}^{-1}(\kappa_2 - \eta_i). \end{aligned}$$

Terrane effects are modeled hierarchically using a non-centered parameterization. We introduce

$$\alpha_j = \mu_\alpha + \sigma_\alpha \alpha_j^{\text{raw}}, \quad \alpha_j^{\text{raw}} \sim \text{N}(0, 1), \quad j = 1, \dots, J,$$

with hyperpriors

$$\mu_\alpha \sim \text{N}(0, 5^2), \quad \log \sigma_\alpha \sim \text{N}(0, 1^2).$$

To enforce the ordering constraint on the cutpoints we parameterize

$$\kappa_1 = \kappa_1^{\text{uncon}}, \quad \kappa_2 = \kappa_1^{\text{uncon}} + \exp(\kappa_\Delta),$$

with

$$\kappa_1^{\text{uncon}} \sim \text{N}(0, 5^2), \quad \kappa_\Delta \sim \text{N}(0, 1^2).$$

Let θ collect all model parameters:

$$\theta = \left(\mu_\alpha, \log \sigma_\alpha, \kappa_1^{\text{uncon}}, \kappa_\Delta, \alpha_1^{\text{raw}}, \dots, \alpha_J^{\text{raw}} \right).$$

The joint log-density $\log p(\theta, y)$ is implemented in **JAX**, and posterior inference proceeds via HMC and NUTS as described next.

Hamiltonian Monte Carlo and the No-U-Turn Sampler

We consider Bayesian inference for a parameter vector $\theta \in \mathbb{R}^d$ with posterior density

$$\pi(\theta) \propto \exp\{\ell(\theta)\} \quad \text{where} \quad \ell(\theta) = \log p(y \mid \theta) + \log p(\theta)$$

is the log joint density of data y and parameters θ , up to an additive constant. Hamiltonian Monte Carlo augments θ with an auxiliary momentum variable $p \in \mathbb{R}^d$ and defines a joint density

$$\tilde{\pi}(\theta, p) \propto \exp\{-H(\theta, p)\},$$

where the Hamiltonian has the standard additive form

$$H(\theta, p) = U(\theta) + K(p), \quad U(\theta) = -\ell(\theta), \quad K(p) = \frac{1}{2}p^\top M^{-1}p,$$

with M a symmetric positive definite mass matrix. The marginal of $\tilde{\pi}(\theta, p)$ in θ is the desired posterior $\pi(\theta)$.

Hamiltonian dynamics and leapfrog integration. The continuous-time Hamiltonian dynamics for $(\theta(t), p(t))$ are defined by

$$\begin{aligned} \frac{d\theta}{dt} &= \nabla_p H(\theta, p) = M^{-1}p, \\ \frac{dp}{dt} &= -\nabla_\theta H(\theta, p) = -\nabla_\theta U(\theta) = \nabla_\theta \ell(\theta). \end{aligned}$$

These differential equations preserve the Hamiltonian, and the associated flow map $(\theta(0), p(0)) \mapsto (\theta(t), p(t))$ is volume-preserving and reversible. If we could simulate this flow exactly, then starting from (θ, p) drawn from $\tilde{\pi}$, running the dynamics for time $T > 0$ and mapping to (θ^*, p^*) would give a new state that also has distribution $\tilde{\pi}$ and hence $\theta^* \sim \pi$ marginally. In practice we discretize Hamiltonian dynamics using a symplectic integrator. The standard choice is the leapfrog method with step size $\varepsilon > 0$:

$$\begin{aligned} p\left(t + \frac{\varepsilon}{2}\right) &= p(t) + \frac{\varepsilon}{2}\nabla_\theta \ell(\theta(t)), \\ \theta(t + \varepsilon) &= \theta(t) + \varepsilon M^{-1}p\left(t + \frac{\varepsilon}{2}\right), \\ p(t + \varepsilon) &= p\left(t + \frac{\varepsilon}{2}\right) + \frac{\varepsilon}{2}\nabla_\theta \ell(\theta(t + \varepsilon)). \end{aligned}$$

A single leapfrog step defines a deterministic, volume-preserving, reversible map

$$(\theta, p) \mapsto \Phi_\varepsilon(\theta, p).$$

Composing L leapfrog steps yields a proposal

$$(\theta^*, p^*) = \Phi_\varepsilon^{(L)}(\theta, p),$$

which approximates the exact Hamiltonian flow at time $T = L\varepsilon$.

Because leapfrog integration introduces discretization error, the Hamiltonian is not exactly conserved and (θ^*, p^*) is not exactly distributed according to $\tilde{\pi}$. To correct this bias, HMC uses a Metropolis–Hastings accept/reject step with acceptance probability

$$\alpha((\theta, p), (\theta^*, p^*)) = \min\{1, \exp(-H(\theta^*, p^*) + H(\theta, p))\}.$$

Basic HMC algorithm. Given the current position $\theta^{(m)}$, one HMC iteration proceeds as:

1. Draw a new momentum

$$p^{(m)} \sim \mathcal{N}(0, M).$$

2. Starting from $(\theta^{(m)}, p^{(m)})$, perform L leapfrog steps of size ε , obtaining proposal $(\theta^*, p^*) = \Phi_\varepsilon^{(L)}(\theta^{(m)}, p^{(m)})$.

3. Accept the proposal with probability

$$\alpha = \min \left\{ 1, \exp(-H(\theta^*, p^*) + H(\theta^{(m)}, p^{(m)})) \right\},$$

and set $\theta^{(m+1)} = \theta^*$ if the move is accepted and $\theta^{(m+1)} = \theta^{(m)}$ otherwise.

The momentum p is discarded after each iteration. The resulting Markov chain $\{\theta^{(m)}\}_{m \geq 0}$ has $\pi(\theta)$ as its invariant distribution. The efficiency of HMC depends strongly on ε , L , and M ; optimal values depend on the geometry of π and are not known in advance [2, 3].

The No-U-Turn Sampler. NUTS removes the need to pre-specify the integration time $T = L\varepsilon$. Starting from a current state θ , NUTS draws a momentum

$$p \sim \mathcal{N}(0, M)$$

and defines the joint log density

$$\log \tilde{\pi}(\theta, p) = \ell(\theta) - \frac{1}{2} p^\top M^{-1} p + \text{const.}$$

An auxiliary slice variable

$$u \sim \text{Uniform}(0, \exp\{\log \tilde{\pi}(\theta, p)\})$$

defines a slice

$$\mathcal{S} = \{(\theta', p') : \exp\{\log \tilde{\pi}(\theta', p')\} \geq u\}.$$

NUTS constructs a trajectory by recursively building a balanced binary tree of states using leapfrog integration. At each doubling step, the algorithm chooses a direction $v \in \{-1, +1\}$ and integrates either forward ($v = +1$) or backward ($v = -1$) in time using leapfrog steps of size ε . Let (θ^-, p^-) and (θ^+, p^+) denote the backward and forward ends of the current trajectory segment, and let $\{\theta_j, p_j\}_{j=1}^K$ be the collection of candidate states generated so far. The no-U-turn stopping rule monitors whether the trajectory begins to turn back. One sufficient condition used in practice is

$$(\theta^+ - \theta^-)^\top p^- < 0 \quad \text{or} \quad (\theta^+ - \theta^-)^\top p^+ < 0,$$

which indicates that continued integration is likely to retrace part of the path rather than explore new regions. When this condition holds, the current subtree is declared complete and no further doubling occurs in that direction. NUTS maintains at each node (i) the leftmost and rightmost states, (ii) the set of candidate states $\mathcal{C} \subset \mathcal{S}$ generated so far, (iii) a flag indicating whether the subtree is valid (no U-turn has occurred and all internal states satisfy

the slice constraint), and (iv) the sum of candidate weights needed for multinomial selection. After the tree-building phase is complete, NUTS selects the next position $\theta^{(m+1)}$ from $\mathcal{C}$, for example by multinomial sampling with probabilities proportional to $\exp\{\log \tilde{\pi}(\theta_j, p_j)\}$ restricted to $\mathcal{S}$.

Let ε denote the step size and choose a target mean acceptance probability $\delta \in (0, 1)$ (for example $\delta = 0.8$). During warmup, ε is updated using a primal–dual averaging scheme of the form

$$\begin{aligned} h_{t+1} &= (1 - \eta_t)h_t + \eta_t(\delta - \alpha_t), \\ \log \varepsilon_{t+1} &= \mu - \frac{\sqrt{t}}{\gamma} h_{t+1}, \end{aligned}$$

where α_t is the observed average acceptance probability at iteration t , η_t is a decaying learning rate, and μ, γ are tuning constants. This scheme drives the step size toward a value that achieves the desired average acceptance rate.

The mass matrix M controls the scaling and correlation structure of the momentum. In a diagonal parameterization, we set $M = \text{diag}(\sigma_1^2, \dots, \sigma_d^2)$. During warmup, NUTS estimates marginal posterior variances of each component of θ and updates M accordingly. Given a batch of samples $\{\theta^{(m)}\}$, one can estimate the empirical covariance matrix $\hat{\Sigma}$ and set $M^{-1} = \hat{\Sigma}$ (full mass matrix) or use only the diagonal elements of $\hat{\Sigma}$ (diagonal mass matrix).

In our implementation we use a diagonal mass matrix, estimated from warmup draws, so that

$$K(p) = \frac{1}{2} \sum_{k=1}^d \frac{p_k^2}{\hat{\sigma}_k^2},$$

which approximately standardizes posterior scales and allows NUTS to use a stable step size. After warmup, the adapted ε and M are held fixed, and NUTS is run with these values to generate posterior samples. Under regularity conditions, the resulting Markov chain is ergodic with invariant distribution $\pi(\theta)$, and the adaptation does not bias the stationary distribution because it is confined to the initial warmup iterations.

NUTS implementation in JAX

We implement the joint log-density and NUTS sampler using JAX and `blackjax`. The model is encoded as a function `radon_risk_log_prob(theta, data)` that returns the joint log-density $\log p(\theta, y)$ for a given parameter vector and data dictionary. The log-density is written in terms of `jax.numpy` operations to allow automatic differentiation.

We run a single NUTS chain with 2,000 warmup iterations for adaptation followed by 4,000 post-warmup iterations, keeping every iteration. The adapted step size is approximately 0.0017 and the inverse mass matrix has dimension 70 (four hyperparameters plus $J = 66$

terrane effects). The mean acceptance probability is approximately 0.92, which is typical for NUTS in moderate dimension and indicates that the integrator step size is neither too large nor too small [3].

Results

Radon rank distribution and geologic context

The marginal distribution of `FIN_RN_RNK` is highly unbalanced: approximately 78% of polygons are rank 1, 16% are rank 2, and 6% are rank 3. This distribution is shown in Figure 1.

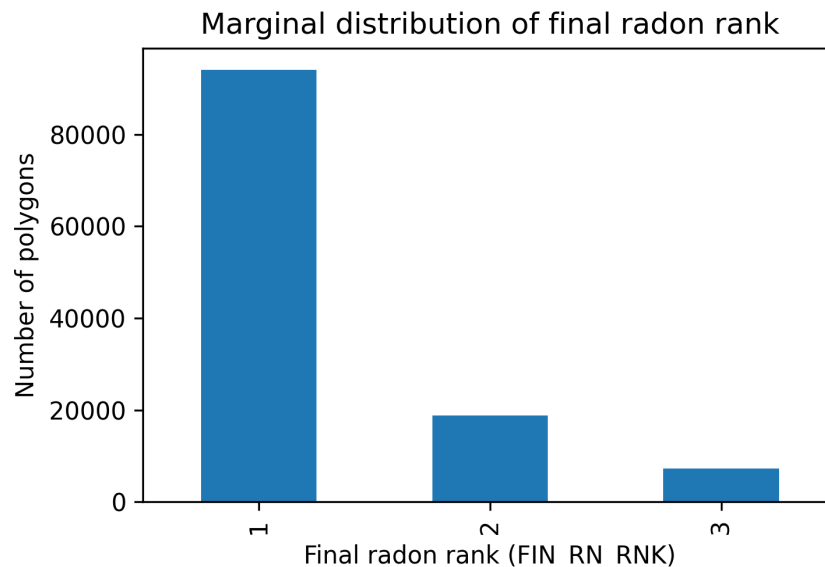


Figure 1: Marginal distribution of the final radon rank `FIN_RN_RNK` across all polygons. Most polygons are classified as lowest risk (rank 1), with relatively few in the highest risk category (rank 3).

Radon potential varies with geologic terrane. There are $J = 66$ distinct terrane groups in the `TERRANE_GR` variable. Figure 2 displays the distribution of `FIN_RN_RNK` for the eight most common terranes. Some terranes are almost entirely in rank 1, while others show a higher proportion of rank 2 and rank 3 polygons. These patterns motivate the hierarchical model with terrane-specific effects on radon risk.

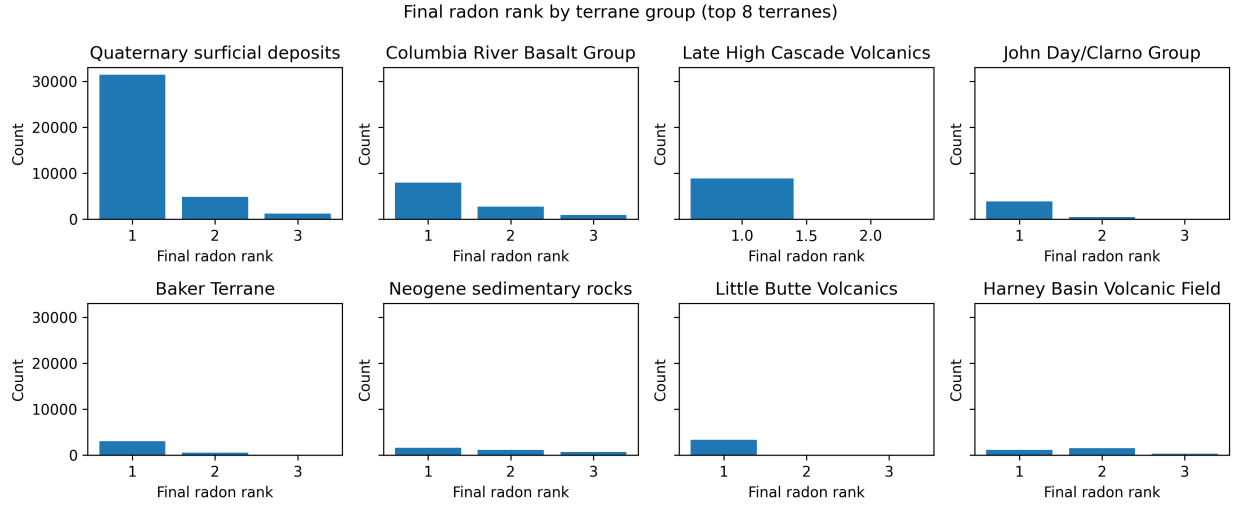


Figure 2: Final radon rank distribution for the eight terranes with the largest numbers of polygons. Each panel shows counts of polygons in radon ranks 1–3 for a single terrane group. The terranes differ in the proportion of higher radon ranks.

MCMC diagnostics

Figures 3–6 show trace plots and marginal posterior histograms for the four hyperparameters. The traces mix well and show no visible nonstationarity. The posterior summaries are

$$\mu_\alpha \approx -1.38 \text{ (sd} = 3.36\text{)}, \quad \sigma_\alpha \approx 2.28 \text{ (sd} = 0.24\text{)},$$

$$\kappa_1 \approx 0.98 \text{ (sd} = 3.37\text{)}, \quad \kappa_2 \approx 2.89 \text{ (sd} = 3.37\text{)}.$$

Approximate effective sample sizes are on the order of 100–500 and split $\hat{R}$ values are close to one for all four hyperparameters, indicating that the NUTS chain provides a reasonable effective sample for these parameters.

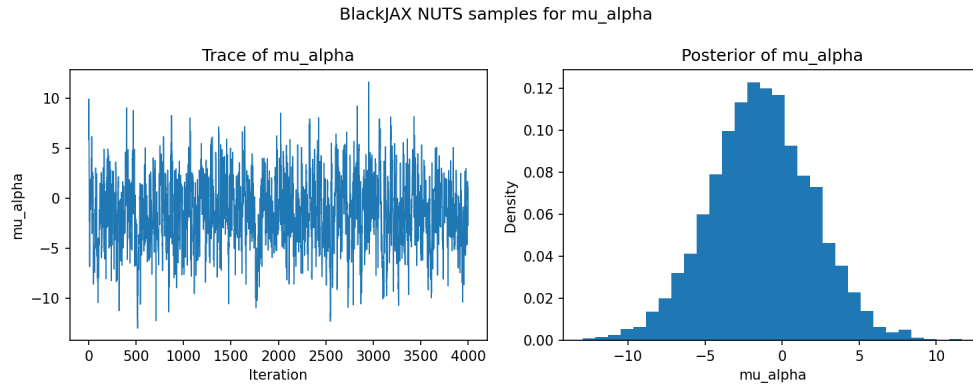


Figure 3: Trace plot and marginal posterior density for the global mean terrane effect μ_α from the `blackjax` NUTS chain.

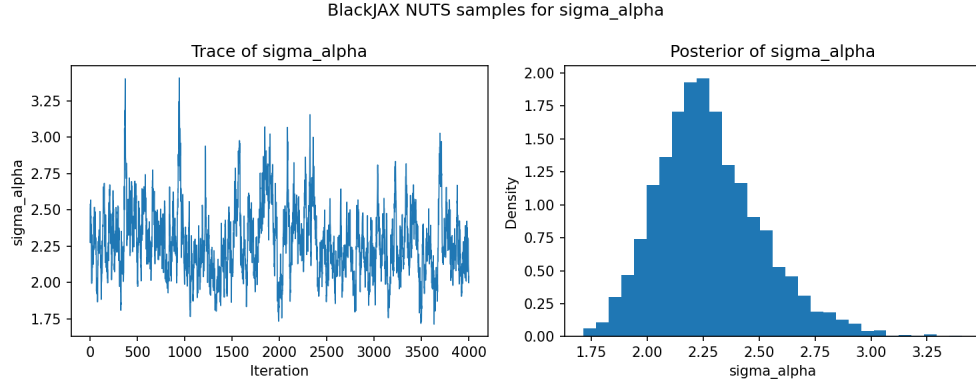


Figure 4: Trace plot and marginal posterior density for the standard deviation of terrane effects σ_α .

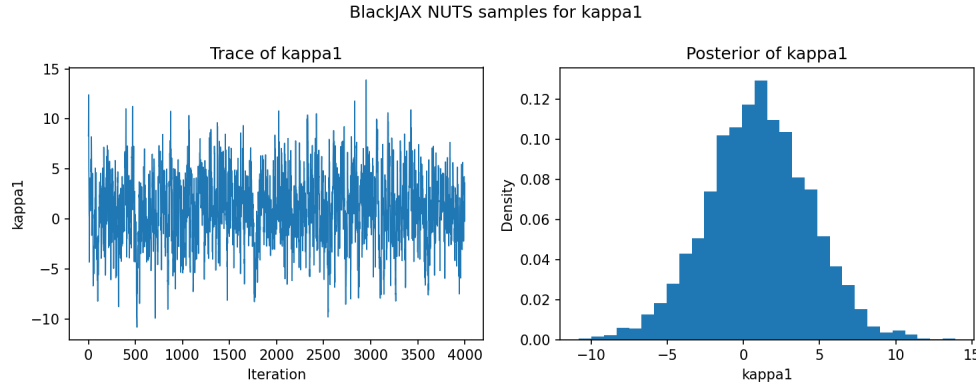


Figure 5: Trace plot and marginal posterior density for the first ordered cutpoint κ_1 .

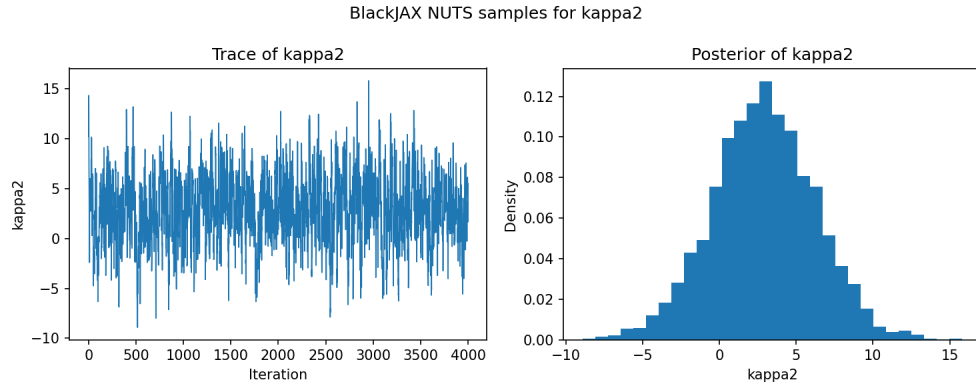


Figure 6: Trace plot and marginal posterior density for the second ordered cutpoint κ_2 .

Figure 7 shows posterior means and 95% credible intervals for the terrane intercepts α_j . The intervals are wide, reflecting substantial uncertainty, but there is clear heterogeneity among terranes. Some terranes have posterior means near or above zero (higher radon ranks) while others are substantially negative (lower radon ranks). The hierarchical prior shrinks all

terrane effects toward the global mean, but terranes with many polygons still show distinct shifts.

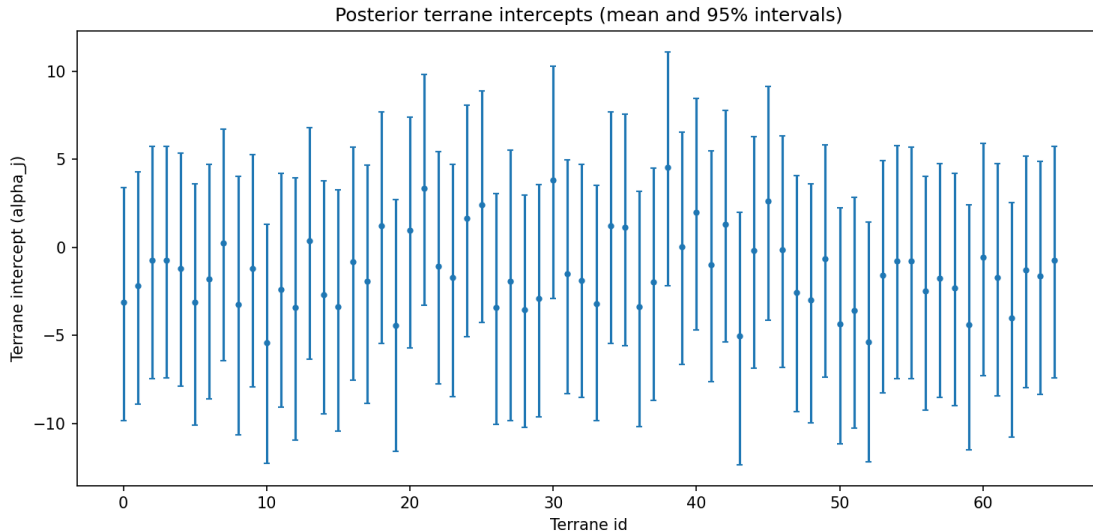


Figure 7: Posterior means and 95% credible intervals for the terrane intercepts α_j from the hierarchical ordered logit model. Terranes are indexed by their integer `terrane_id`.

Terrane-level radon risk

From the posterior draws of $(\alpha_j, \kappa_1, \kappa_2)$ we compute the implied terrane-level probabilities

$$p_{j,c} = \Pr(y = c \mid \alpha_j, \kappa_1, \kappa_2), \quad c = 1, 2, 3.$$

For each terrane we summarize the posterior means $\mathbb{E}[p_{j,c} \mid y]$. As an example, one terrane (Columbia River Basalt Group) has posterior mean probabilities approximately $(p_{j,1}, p_{j,2}, p_{j,3}) = (0.68, 0.26, 0.07)$, indicating a higher chance of ranks 2 and 3 compared with the overall marginal distribution.

Figure 8 displays the distribution of posterior mean $p_{j,3}$ across all $J = 66$ terranes. Most terranes have posterior mean $p_{j,3}$ close to zero, but a small number of terranes have values above 0.2 and a few approach 0.8, suggesting that high radon risk is concentrated in a limited set of geologic settings.

Posterior predictive assessment

To check whether the model reproduces the overall distribution of radon ranks, we draw posterior predictive samples. For each saved draw of $(\alpha_j, \kappa_1, \kappa_2)$ we generate a new radon

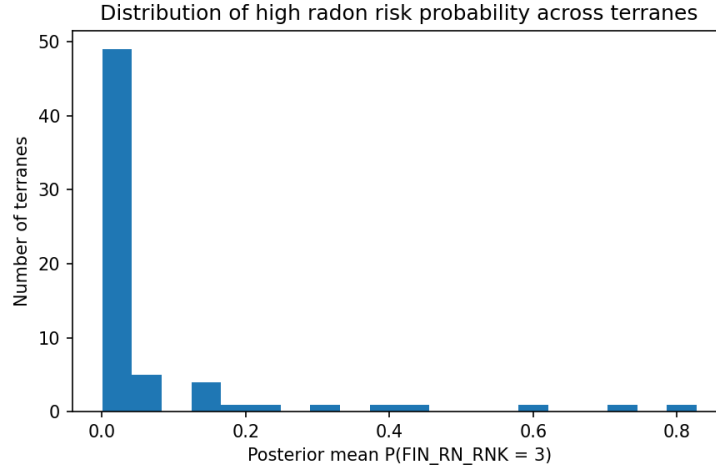


Figure 8: Histogram of terrane-level posterior mean probabilities $p_{j,3} = \Pr(\text{FIN_RN_RNK} = 3)$ under the hierarchical model. Most terranes have low estimated probability of being in the highest radon-risk category, while a few terranes have much larger values.

rank $\tilde{y}_i$ for every polygon by sampling from the ordered logistic probabilities, and compute the proportion of $\tilde{y}_i$ in each category. This yields a posterior distribution for the marginal probabilities $\Pr(\tilde{y} = c)$, $c = 1, 2, 3$.

Figure 9 compares the observed proportions to the posterior predictive means. The model closely matches the empirical marginal distribution: for rank 1 the observed proportion is 0.784 and the posterior predictive mean is 0.784 (with a 95% interval of approximately $[0.782, 0.785]$), and similar agreement holds for ranks 2 and 3. This suggests that the hierarchical ordinal model captures the overall imbalance among the three radon categories.

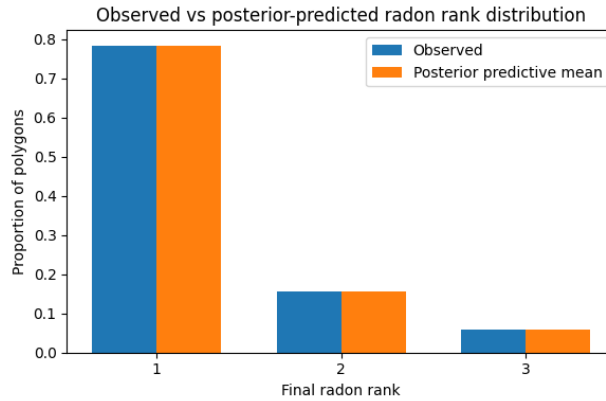


Figure 9: Posterior predictive check for the marginal distribution of `FIN_RN_RNK`. Bars show the observed proportions of polygons in each radon rank (blue) and the posterior predictive mean proportions (orange). The posterior predictive distribution closely matches the empirical marginal distribution.

Discussion

This analysis shows that a hierarchical ordinal model fit with NUTS can recover the observed marginal distribution of radon potential ranks and identify terranes with elevated risk. A key strength of the approach is partial pooling across geologic groups, which stabilizes estimates for terranes with few polygons while allowing high-information terranes to deviate from the overall mean. Another strength is the ability to propagate uncertainty in terrane effects into posterior distributions of high-risk probabilities, which is important when using these estimates to inform screening or mitigation decisions.

There are several limitations. The model uses only terrane-level intercepts and does not explicitly incorporate spatial correlation or additional covariates such as rock type, radiometric measurements, or indoor radon measurement averages beyond what is encoded in the final radon rank. As a result, some local spatial structure in radon risk may be averaged out. In addition, we analyze the DOGAMI radon rank rather than raw indoor concentration data, so our inference is conditional on the agency’s ranking procedure and cannot directly recover absolute radon concentrations.

Future work could extend the model to include spatial random effects, additional geologic predictors, or household-level measurement data where available. It would also be natural to compare NUTS with alternative gradient-based samplers or variational approximations in this setting, to evaluate tradeoffs between computational cost and accuracy. Finally, one could integrate the resulting terrane-level posterior probabilities into decision analyses that weigh the costs and benefits of targeted radon testing or mitigation across Oregon.

Conclusion

We fit a hierarchical ordered logistic regression model to the Oregon DOGAMI radon potential dataset using Hamiltonian Monte Carlo with the No-U-Turn Sampler implemented in **JAX**. The NUTS algorithm, with automatic tuning of step size and mass matrix, mixed well in this posterior and produced effective sample sizes on the order of hundreds for key hyperparameters. Posterior predictive checks indicate that the model reproduces the observed marginal distribution of radon potential ranks.

At the application level, the model yields terrane-level posterior probabilities for each radon rank and shows that high radon potential is concentrated in a small subset of terranes. These estimates could inform prioritization of radon testing and mitigation efforts in Oregon by highlighting geologic settings with elevated probability of the highest risk category, while accounting for uncertainty in the underlying parameters. More broadly, the analysis illustrates that NUTS in **JAX** provides a practical and effective tool for fitting hierarchical ordinal models at the scale of tens of thousands of observations and dozens of groups.

References

- [1] Hoffman, M. D. and Gelman, A. (2014). The No-U-Turn sampler: Adaptively setting path lengths in Hamiltonian Monte Carlo. *Journal of Machine Learning Research*, **15**, 1593–1623. (arXiv:1111.4246).
- [2] Neal, R. M. (2011). MCMC using Hamiltonian dynamics. In Brooks, S., Gelman, A., Jones, G. L., and Meng, X.-L. (eds.), *Handbook of Markov Chain Monte Carlo*, pp. 113–162. Chapman & Hall/CRC, Boca Raton.
- [3] Betancourt, M. (2017). A conceptual introduction to Hamiltonian Monte Carlo. *arXiv preprint*, arXiv:1701.02434.
- [4] Gelman, A., Carlin, J. B., Stern, H. S., Dunson, D. B., Vehtari, A., and Rubin, D. B. (2013). *Bayesian Data Analysis*, 3rd edition. Chapman & Hall/CRC, Boca Raton.
- [5] Vehtari, A., Gelman, A., Simpson, D., Carpenter, B., and Bürkner, P.-C. (2021). Rank-normalization, folding, and localization: An improved $\hat{R}$ for assessing convergence of MCMC. *Bayesian Analysis*, **16**(2), 667–718.
- [6] Bradbury, J., Frostig, R., Hawkins, P., et al. (2018). JAX: composable transformations of Python+NumPy programs. Available at <https://github.com/google/jax>.
- [7] Launay, Y., Louf, R., and contributors (2020–). BlackJAX: A sampling library for JAX. Available at <https://github.com/blackjax-devs/blackjax>.
- [8] Oregon Department of Geology and Mineral Industries (DOGAMI) (2018). Radon potential of Oregon (polygon feature class). Open-File Report O-18-01. Available from Oregon Department of Geology and Mineral Industries data portal.
- [9] McCullagh, P. (1980). Regression models for ordinal data. *Journal of the Royal Statistical Society, Series B*, **42**(2), 109–142.

Appendix

A.1 MCMC configuration

We fit the hierarchical ordered logit model using Hamiltonian Monte Carlo with the No-U-Turn Sampler implemented in `blackjax` in `JAX`. The main configuration was:

- Number of polygons: $N = 120,049$.
- Number of terranes: $J = 66$.
- One NUTS chain with 2,000 warmup iterations and 4,000 post-warmup iterations.
- Adapted step size: $\varepsilon \approx 0.00172$.
- Inverse mass matrix dimension: 70 (four hyperparameters plus $J = 66$ terrane effects).
- Mean acceptance rate: $\bar{\alpha} \approx 0.92$.

The marginal distribution of the ordinal outcome `FIN_RN_RNK` in the analysis dataset was

$$\text{count}(1, 2, 3) = (94,071, 18,788, 7,190),$$

corresponding to observed proportions (0.784, 0.157, 0.060).

A.2 Posterior summaries and diagnostics

Table 1 reports posterior means and standard deviations for the main hyperparameters and the pooled terrane intercepts, based on the 4,000 post-warmup NUTS draws.

Table 1: Posterior summaries for selected parameters from the hierarchical ordered logit model.

Parameter	Mean	SD
μ_α	-1.377	3.364
σ_α	2.277	0.235
κ_1	0.976	3.365
κ_2	2.887	3.365
α (all terranes, pooled)	-1.389	4.052

We also computed effective sample sizes (ESS) using an initial positive sequence autocorrelation estimator and split- $\hat{R}$ by dividing the single chain into four segments. The diagnostics for the four hyperparameters are shown in Table 2.

For the terrane-specific intercepts α_j , the first five terranes (in numeric `terrane_id` order) all had ESS values around 480 and split- $\hat{R}$ values between 1.005 and 1.006, indicating good mixing for both global and group-level parameters.

Table 2: MCMC diagnostics for key hyperparameters: effective sample size (ESS) and split- $\hat{R}$.

Parameter	ESS	split- $\hat{R}$
μ_α	≈ 476	1.004
σ_α	≈ 101	1.016
κ_1	≈ 478	1.005
κ_2	≈ 478	1.006

A.3 Posterior predictive check for marginal rank distribution

Using the posterior draws of $(\alpha_j, \kappa_1, \kappa_2)$, we generated posterior predictive replicate datasets and computed the marginal proportions of ranks $\tilde{y} \in \{1, 2, 3\}$ at each draw. Table 3 summarizes the observed proportions and the posterior predictive means with 95% central intervals.

Table 3: Posterior predictive check for the marginal distribution of FIN_RN_RNK.

Rank	Observed proportion	Posterior predictive mean	95% posterior interval
1	0.784	0.784	[0.782, 0.785]
2	0.157	0.157	[0.155, 0.158]
3	0.060	0.060	[0.059, 0.061]

The close agreement between the observed and posterior predictive proportions supports the conclusion that the hierarchical ordinal model reproduces the general imbalance between the three radon risk categories.

A.4 Core JAX/blackjax implementation

The code below defines the joint log-density in JAX and runs NUTS via `blackjax`:

```
import numpy as np
import pandas as pd
import jax
import jax.numpy as jnp
import blackjax

LOG_2PI = jnp.log(2.0 * jnp.pi)

def normal_logpdf(x, loc, scale):
    z = (x - loc) / scale
    return -0.5 * (z**2 + 2.0 * jnp.log(scale) + LOG_2PI)

def ordered_loglik(y, eta, kappa1, kappa2):
    sigmoid = jax.nn.sigmoid
```

```

    c1 = sigmoid(kappa1 - eta)
    c2 = sigmoid(kappa2 - eta)
    p1 = jnp.clip(c1, 1e-12, 1.0 - 1e-12)
    p2 = jnp.clip(c2 - c1, 1e-12, 1.0 - 1e-12)
    p3 = jnp.clip(1.0 - c2, 1e-12, 1.0 - 1e-12)
    logp = jnp.where(y == 1, jnp.log(p1),
                     jnp.where(y == 2, jnp.log(p2), jnp.log(p3)))
    return jnp.sum(logp)

def unpack_theta_risk(theta, n_groups):
    mu_alpha = theta[0]
    log_sigma_alpha = theta[1]
    kappa1_uncon = theta[2]
    kappa_delta = theta[3]
    alpha_raw = theta[4:]
    return mu_alpha, log_sigma_alpha, kappa1_uncon, kappa_delta, alpha_raw

def radon_risk_log_prob(theta, data):
    theta = jnp.asarray(theta)
    y = data["y"]
    terrane_idx = data["terrane_idx"]
    n_groups = int(data["n_groups"])

    (mu_alpha,
     log_sigma_alpha,
     kappa1_uncon,
     kappa_delta,
     alpha_raw) = unpack_theta_risk(theta, n_groups)

    sigma_alpha = jnp.exp(log_sigma_alpha)
    kappa1 = kappa1_uncon
    kappa2 = kappa1_uncon + jnp.exp(kappa_delta)
    alpha = mu_alpha + sigma_alpha * alpha_raw

    lp = 0.0
    lp += normal_logpdf(mu_alpha, 0.0, 5.0)
    lp += normal_logpdf(log_sigma_alpha, 0.0, 1.0)
    lp += normal_logpdf(kappa1_uncon, 0.0, 5.0)
    lp += normal_logpdf(kappa_delta, 0.0, 1.0)
    lp += jnp.sum(normal_logpdf(alpha_raw, 0.0, 1.0))

    eta = alpha[terrane_idx]
    lp += ordered_loglik(y, eta, kappa1, kappa2)
    return lp

# --- NUTS setup and sampling ---

df = pd.read_csv("Radon_Data_1136320385238331420.csv")
df = df.dropna(subset=["TERRANE_GR", "FIN_RN_RNK"]).reset_index(drop=True)

df["terrane_cat"] = df["TERRANE_GR"].astype("category")
df["terrane_id"] = df["terrane_cat"].cat.codes

y = df["FIN_RN_RNK"].astype(int).values

```

```

terrane_idx = df["terrane_id"].values
n_groups    = df["terrane_id"].nunique()

data = dict(
    y=jnp.array(y, dtype=jnp.int32),
    terrane_idx=jnp.array(terrane_idx, dtype=jnp.int32),
    n_groups=n_groups,
)

dim      = 4 + n_groups
theta0   = jnp.zeros(dim)

def logprob(theta):
    return radon_risk_log_prob(theta, data)

num_warmup   = 2000
num_samples  = 4000

key = jax.random.PRNGKey(0)
key, warmup_key = jax.random.split(key)

adapt = blackjax.window_adaptation(blackjax.nuts, logprob)
(last_state, parameters), _ = adapt.run(warmup_key, theta0, num_warmup)

nuts = blackjax.nuts(
    logprob,
    step_size=parameters["step_size"],
    inverse_mass_matrix=parameters["inverse_mass_matrix"]
)
kernel = jax.jit(nuts.step)

key, sample_key = jax.random.split(key)
keys = jax.random.split(sample_key, num_samples)

def one_step(state, key):
    state, info = kernel(key, state)
    return state, (state, info)

_, (states, infos) = jax.lax.scan(one_step, last_state, keys)
samples = states.position

```
